Einführung in die hardwarenahe Programmierung

Andreas Scheibenpflug MSc.
andreas.scheibenpflug@fh-hagenberg.at

Bachelor-Studiengang
Software Engineering

Fakultät für Informatik, Kommunikation und Medien

Hagenberg, FH-OÖ

1. Hardware
   - Begriffe
   - STM32
2. Software
   - Unterschiede in der Programmierung
   - Stufen des Softwaresupports
3. Literatur

Inhalt

1 Hardware
- Begriffe
- STM32

2 Software

3 Literatur

Embedded Systems

Aus *Making embedded systems*:

'An embedded system is a computerized system that is purpose-built for its application.'

- Ein eingebettetes System ist für eine bestimmte Aufgabe gebaut
- Die Software auf diesem System wurde ebenfalls für genau diese eine Aufgabe entwickelt
- Ein Computer kann im Gegensatz dazu unterschiedlichste Aufgaben erledigen
- Warum heißen diese Systeme *eingebettete Systeme*?
- Warum werden nicht einfach Computer verwendet?

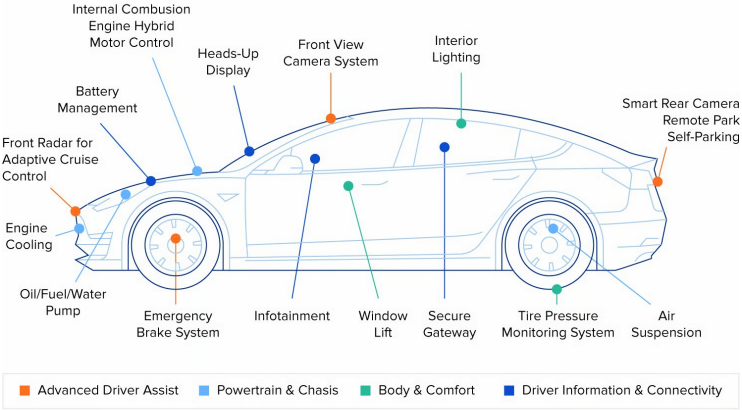
Einsatzgebiete







Figure 3: Automotive Embedded Systems



toptal

Microcontroller vs. Microprozessor

- Microcontroller (microcontroller unit (MCU)) besteht u.A. aus einem Prozessor, Speicher und Peripherie
- In einem Chip integriert, ähnlich einem System-on-a-Chip (SoC) Design
- Microprozessor besteht aus einem Prozessor mit zusätzlicher Funktionalität
  - Erweiterter Befehlssatz
  - Zusätzliche Komponenten, z.B. MMU, TLB, Protection Rings,...
  - $\rightarrow$ Unterstützung für Betriebssysteme
  - Leistungsfähiger, mehr Kerne
  - Flexibler einsetzbar, komplexer
  - Aber: teurer, höherer Energieverbrauch





STMicroelectronics

- Größter europäischer Halbleiterhersteller mit Sitz in der Schweiz
- Produziert eine breite Palette von Halbleiterprodukten, darunter Mikrocontroller, Sensoren, Leistungshalbleiter und integrierte Schaltungen
- Bekannt für seine STM32-Serie von Mikrocontrollern, die auf der ARM-Architektur basieren
- Diese Mikrocontroller werden in einer Vielzahl von Anwendungen eingesetzt, darunter Industrieautomation, Konsumgüter, Automobilindustrie und Internet der Dinge (IoT)
- $\rightarrow$ Sehr weit verbreitet, keine 'Bastlerhardware'



ARM

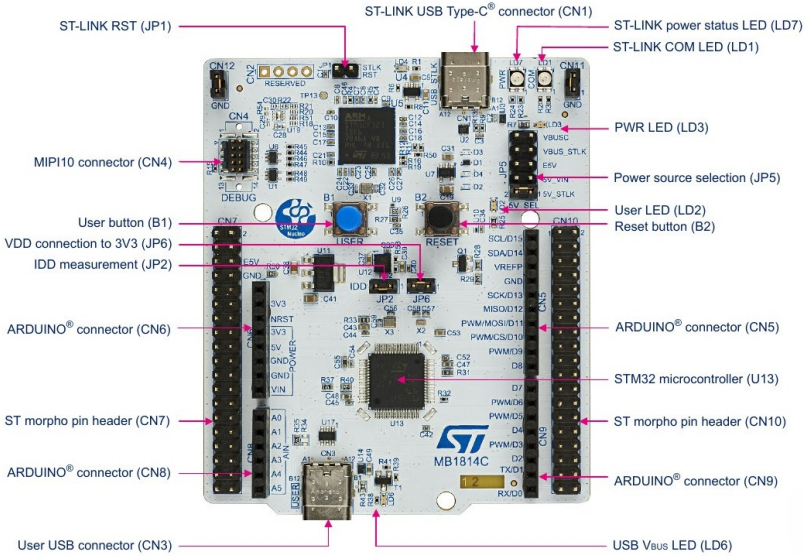
- ARM ist ein britisches Unternehmen, das Mikroprozessorarchitekturen entwirft
- ARM entwickelt nur das Instruction Set und lizensiert es an andere Unternehmen
- ARM-Prozessoren sind bekannt für ihre Energieeffizienz (und Leistung)
- Weit verbreitet in mobilen Geräten, eingebetteten Systemen und IoT-Anwendungen
- Zunehmend auch im Desktopbereich
- Cortex-M Serie ist speziell für Mikrocontroller entwickelt
- Cortex-A Serie ist für Mobile/Desktop/Server Anwendungen entwickelt (besitzt MMU)



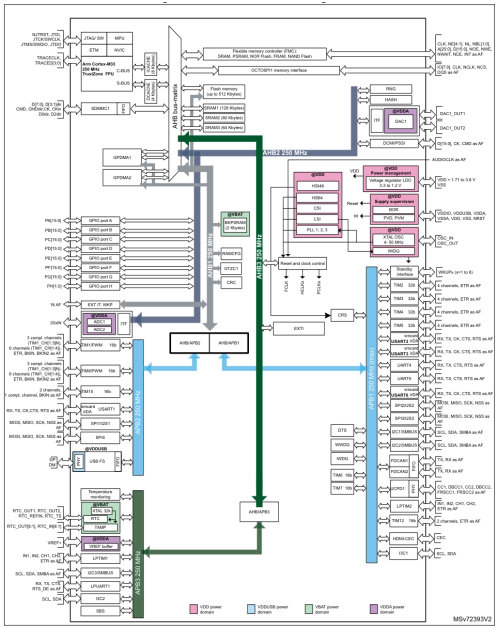
Nucleo Boards

- Nucleo Boards sind eine Reihe von Evaluierungsboards von ST
- In unterschiedlichen Konfigurationen verfügbar
- Werden zum Entwickeln und Testen verwendet bevor die tatsächliche Hardware entwickelt/gekauft wird
- Enthalten einen HW Debug Chip für ST-Link
- Nucleo STM32H533RE: 'High-Performance'
  - ARM Cortex-M33 MCU
  - Max. 250 MHz CPU (in Übung: 240 MHz)
  - 512-Kbyte Flash
  - 272-Kbyte RAM
  - Div. Ports, Busse, Clocks, GPIOs, Timer, UARTs, A/D - D/A Wandler, ...

Nucleo STM32H533RE



Nucleo STM32H533RE Blockdiagram



Nucleo Safety recommendations

Aus dem User Manual, targeted audience:
This product targets users with at least basic electronics or embedded software development knowledge such as engineers, technicians, or students. This board is not a toy and is not suited for use by children.
Handling the board:

- The connection pins on the board might be sharp. Be careful when handling the board to avoid hurting yourself
- This board contains static-sensitive devices. To avoid damaging it, handle the board in an ESD-proof environment.
- While powered, do not touch the electric connections on the board with your fingers or anything conductive. The board operates at a voltage level that is not dangerous, but components might be damaged when shorted.
- Do not put any liquid on the board and avoid operating the board close to water or at a high humidity level.
- Do not operate the board if dirty or dusty

→ Behandeln Sie das Board wie ein rohes Ei!
Wenn Sie das Board beschädigen: Bitte melden!

Beispiel: Teufel Mynd

- Teufel Mynd
- Open Source Bluetooth Lautsprecher
- `https://github.com/teufelaudio/mynd-firmware`
- FreeRTOS
- STM32F0 series (max. 48 Mhz, 32 Kbyte RAM)



Inhalt

1 Hardware

2 Software
- Unterschiede in der Programmierung
- Stufen des Softwaresupports

3 Literatur

Softwareentwicklung für Microcontroller

- Erfordert Kenntnisse in hardwarenaher Programmierung und der Hardware
- Direkter Zugriff auf Hardware und Register (Vergleich mit BSY: 'Kernel Space')
- Ressourcen sind eingeschränkt (CPU, Memory, Befehlssatz $\rightarrow$ Floating Point Operationen,...)
- Kein Betriebssystem, oder nur ein Echtzeitbetriebssystem
- Keine Sicherheitsnetze wie z.B. virtual/protected Memory
- U.U. andere Entwicklungswerkzeuge (z.B. Cross-Compiler)
- U.U. eingeschränkte Debugging- und Testwerkzeuge
- Eingeschränkte Auswahl an Standardbibliotheken
- Andere Art der Programmierung (z.B. keine dynamische Speicherverwaltung, Superloop,...)
- $\sim$ Es läuft genau ein Programm und sonst nichts

Welcome to Hell



Abbildung: Anwendungsentwicklung mit OS

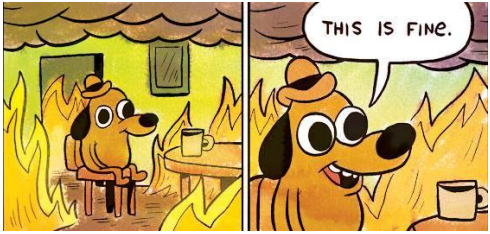


Abbildung: Mikrocontrollerprogrammierung

- Ganz so schlimm ist es natürlich auch nicht... 😉
- Es gibt Bibliotheken und Frameworks, die die Entwicklung erleichtern
- z.B. CMSIS, HAL, BSP, FreeRTOS
- Programme haben oft weniger LOC, übersichtlicher
- Weniger Abhängigkeiten
- Keine dynamische Speicherverwaltung $\rightarrow$ keine Probleme?

Ein erstes Programm

```
void main(void) {
    /* initialize hardware */
    pl_init();

    volatile uint32_t j = 0;

    /* super loop */
    while (1) {
        j++;
    }
}
```

Stufen des Softwaresupports

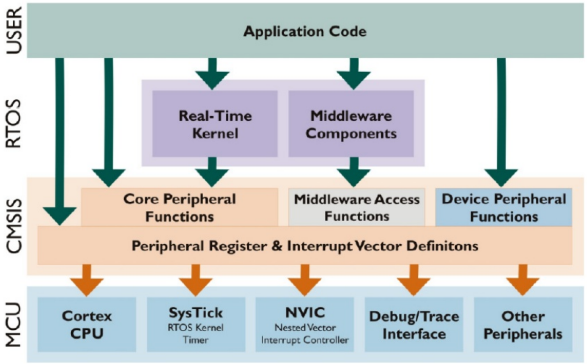


Abbildung: Layering, Endausbaustufe

- Es gibt verschiedene Stufen des Softwaresupports (Layering)
- Diese bauen oft aufeinander auf
- Jede zusätzliche Stufe erleichtert die Entwicklung, erhöht aber auch den Ressourcenverbrauch

Bare Metal

- Es werden keine Bibliotheken verwendet
- Nur ein Cross-Compiler, Linker und ein Tool zum Flashen
- Linker Script, Startup Code, HW-Zugriff muss implementiert werden
- Direkter Zugriff auf Hardware, keine Abstraktion
- Vorteil: Ressourcen-schonend, kleine Anwendungsgröße (wichtig bei kleinem Flash Speicher/RAM)
- Nachteil: Hoher Entwicklungsaufwand, fehlende Portabilität
- $\rightarrow$ wird in der Praxis nicht gemacht

$\rightarrow$ siehe Literatur Bare Metal Programming Guide

CMSIS

- Common Microcontroller Software Interface Standard (CMSIS)
- Bibliothek, welche von ARM entwickelt und zur Verfügung gestellt wird
- Standardisierte, hardwareunabhängige Schnittstelle zu Prozessor und Peripherie
- Verbessert die Portabilität von Software
- Erweiterbar, d.h. es gibt z.B. auch Support für Neuronale Netze
- Konkret: C Header Files, welche `#DEFINE`s und `structs` enthalten

HAL

- Hardware Abstraction Layer (HAL) abstrahiert die Hardware/Peripherie
- Stellt vor allem Funktionen zur Verfügung, die den Umgang mit der Hardware erleichtern
- Abstrahiert viel der Hardwaredetails, ähnlich zu Treibern bei Betriebssystemen
- Erhöht daher wiederrum die Portabilität der Anwendung
- Konkret: C Module für die verbaute Peripherie, also z.B. GPIO, Timer, USB, UART, Bluetooth,...
- Kann sehr groß werden

BSP

- Board Support Package (BSP) enthält Code zum Initialisieren des Prozessors, RAM, und Board
- Konfiguration der Interrupt Tabelle
- Enthält oft auch die Speichersegmentierung
- Kann Teile der C Standardbibliothek und andere Low-Level Hilfsfunktionen enthalten
- Konkret: C Module, Assembler Files, Linker Scripts
- Begriffe werden etwas vermischt, BSP kann auch HAL enthalten

Echtzeitbetriebssystem

- Wird als Bibliothek zum Programm hinzugelinkt
- Ermöglicht das Implementieren von 'Tasks'
- Besitzen einen Scheduler ähnlich einem Betriebssystem, erlaubt Preemption
- Bietet Synchronisationsmechanismen (Mutexe, Semaphoren,...) und Memory Management
- Zusätzlich auch Funktionalität zur Abstrahierung der Hardware
- Vertreter: FreeRTOS oder Zephyr

Zusammenfassend

- Grenzen zwischen den einzelnen Stufen sind oft fließend bzw. überschneiden sich
- In der Praxis werden oft mehrere Stufen kombiniert
- Welche Teile verwendet werden hängt von den Anforderungen an Funktionalität und Kosten ab
- Umso mehr Code inkludiert wird, umso mehr Speicher muss die Hardware bereitstellen und damit steigen die Kosten
- Ein Mittelweg ist, das BSP und den HAL zu adaptieren bzw. Teile selbst zu implementieren
- Dies ist der Weg den wir für die Übungen (plib) gewählt haben und welcher auch in Ralf Jesse's Buch beschrieben wird (siehe Literatur)

Inhalt

1 Hardware

2 Software

3 Literatur

Literatur

- Elecia White, Making Embedded Systems, 2nd edition, O'Reilly, 2024
- Ralf Jesse, STM32: ARM-Mikrocontroller programmieren für Embedded Systems, 2. Auflage, mitp, 2021
- Bare Metal Programming Guide,
  - `https://github.com/cpq/bare-metal-programming-guide`
  - Entwicklung von Bare-Metal-Anwendungen, komplett ohne Hilfsmittel und Bibliotheken
- STM32H533 User- und Reference Manual aus dem eLearning

